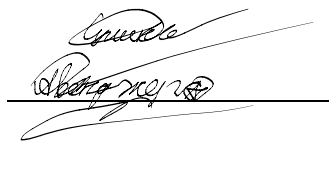


МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра САУ

ОТЧЕТ
по лабораторной работе №7
по дисциплине «Микропроцессорная техника в мехатронике и
робототехнике»
Тема: Обмен данными по протоколу SPI на примере АЦП
Вариант 8

Студенты гр. 2491

Преподаватель



Two handwritten signatures are present, each followed by a horizontal line. The first signature is above the line, and the second is below it.

Сулинский Е.В.
Недовизий А.И.

Копычев М.М.

Санкт-Петербург

2025

Цель работы

Изучение работы с протоколом SPI на примере АЦП микроконтроллера Atmega 128.

Задание на работу

Написать программный код для вывода числовой последовательности на семисегментные индикаторы. Для выполнения лабораторной работы рекомендуется использовать внешние устройства ввода, такие как потенциометр, кнопка или энкодер.

Краткие теоретические сведения

SPI (Serial Peripheral Interface) — это синхронный последовательный интерфейс связи, широко используемый в микроконтроллерах AVR для обмена данными с периферийными устройствами (например, АЦП, ЦАП, дисплеями, памятью и т.д.).

Основные характеристики SPI в AVR:

- Мастер/Ведомый режим: микроконтроллер может работать как ведущим (Master), так и ведомым (Slave).
- Высокая скорость передачи
- Аппаратная реализация: использует встроенный модуль SPI (в некоторых моделях — USI или USART в режиме SPI).

Основные линии интерфейса SPI:

- MOSI (Master Out Slave In) — данные от ведущего к ведомому.
- MISO (Master In Slave Out) — данные от ведомого к ведущему.
- SCK (Serial Clock) — тактовый сигнал, генерируемый ведущим.
- SS (Slave Select) — выбор ведомого устройства (активный низкий уровень).

Примечание: в режиме Slave микроконтроллер AVR использует пин /SS как вход для определения своего состояния (если /SS становится высоким, AVR может выйти из режима Slave, если не заблокирован программно).

Регистры SPI в AVR:

- SPCR (SPI Control Register) — управление режимом работы, скоростью, прерываниями.
- SPSR (SPI Status Register) — содержит флаги состояния, включая скорость передачи (бит SPI2X).
- SPDR (SPI Data Register) — регистр данных для передачи/приёма.

Режимы SPI (полярность и фаза тактового сигнала):

Определяются битами CPOL и CPHA в регистре SPCR:

Особенности работы в AVR:

- В режиме Master пин /SS должен быть настроен как выход (даже если не используется), иначе AVR может случайно перейти в Slave-режим.

- В режиме Slave пин /SS должен быть подтянут к VCC или управляться внешним Master'ом.
- Передача данных инициируется записью байта в SPDR. По завершении флаг SPIF в SPSR устанавливается.

SPI в AVR — простой, быстрый и надёжный способ связи с периферией, особенно при работе с несколькими устройствами через отдельные линии SS.

Ход работы

Напишем программу для реализации алгоритма из задания. Для более наглядного представления выполняемого алгоритма сделаем блок схемы, изображенные на рисунках 1 и 2.

Код программы

```
#include <avr/io.h>
#include <util/delay.h>
#include <avr/interrupt.h>

volatile int16_t value = 0; // текущее значение для вывода
volatile uint8_t last_state; // хранит предыдущее состояние энкодера

void spiInit(void);
void showMe(int16_t spiData);
uint8_t digit(uint16_t d, uint8_t m);

// --- Инициализация SPI ---
void spiInit(void) {
    DDRB |= (1<<2)|(1<<3)|(1<<5); // SS, MOSI, SCK — выходы
    SPCR = (1<<SPE)|(1<<MSTR)|(1<<SPR1)|(1<<SPR0); // мастер,
    делитель /128
}

// --- Прерывание от энкодера (INT0, INT1) ---
ISR(INT0_vect) {
    uint8_t stateA = (PIND >> 2) & 1; // PD2 = канал A
    uint8_t stateB = (PIND >> 3) & 1; // PD3 = канал B

    if (stateA != last_state) {
        if (stateB != stateA)
            value++; // вращение по часовой стрелке
        else
            value--; // против часовой стрелки

        if (value > 999) value = 999;
        if (value < -99) value = -99;

        last_state = stateA;
    }
}

// --- Вспомогательные функции (твоя версия showMe и digit) ---
```

```

uint8_t digit(uint16_t d, uint8_t m) {
    uint8_t i = 3, a;
    while(i) {
        a = d % 10;
        if(i-- == m) break;
        d /= 10;
    }
    return a;
}

```

```

void showMe(int16_t spiData) {
    int8_t i;
    uint8_t dig[] = {0,0,0}, j;
    uint8_t mas[] =
        {0x81,0xF3,0x49,0x61,0x33,0x25,0x05,0xF1,0x01,0x21,0xFF,0x7F};
    uint16_t res;
    if (spiData < 0) res = -spiData;
    else res = spiData;

    for (i = 1; i <= 3; i++)
        dig[i - 1] = digit(res, i);

    if (res != 0) {
        j = 0;
        while (dig[j] == 0 && j < 2)
            j++;

        if (spiData < 0) {
            SPDR = mas[11];
            while (!(SPSR & (1 << SPIF)));
        } else {
            SPDR = mas[10];
            while (!(SPSR & (1 << SPIF)));
        }

        for (i = 0; i <= 2; i++) {
            if (i < j) SPDR = mas[10];
            else SPDR = mas[dig[i]];
            while (!(SPSR & (1 << SPIF)));
        }
    } else {
        for (i = 0; i <= 2; i++) {
            SPDR = mas[10];
            while (!(SPSR & (1 << SPIF)));
        }
    }
}

```

```

        SPDR = mas[0];
        while (!(SPSR & (1 << SPIF)));
    }

    PORTB &= ~(1 << 2);
    _delay_us(20);
    PORTB |= (1 << 2);
    _delay_us(20);
    PORTB &= ~(1 << 2);
}

int main(void) {
    spiInit();

    // --- Настраиваем энкодер (PD2=A, PD3=B) ---
    DDRD &= ~((1 << 2) | (1 << 3)); // входы
    PORTD |= (1 << 2) | (1 << 3); // подтяжка к +V

    // --- Прерывание по изменению состояния канала A (INT0) ---
    EICRA = (1 << ISC00); // по любому изменению INT0
    EIMSK = (1 << INT0); // разрешить INT0

    sei();

    last_state = (PIND >> 2) & 1;

    while (1) {
        showMe(value);
        _delay_ms(50);
    }
}

```

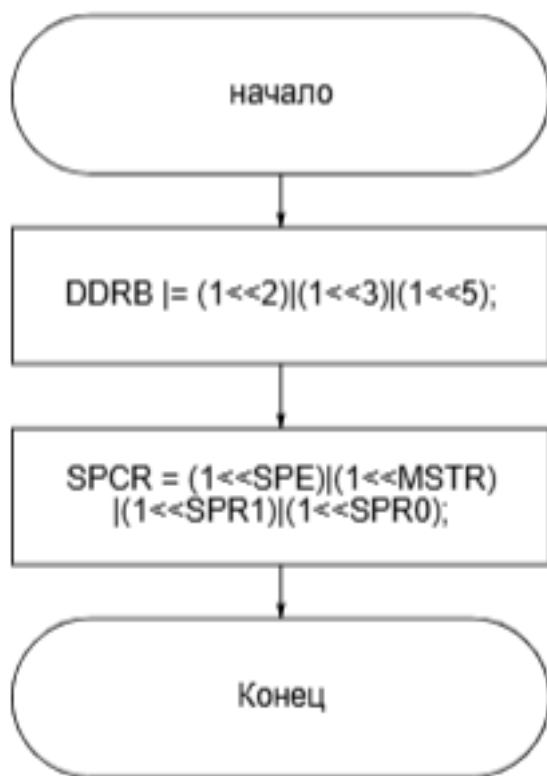


Рисунок 1 – “Блок-схема функции spiInit(void)”



Рисунок 2 – “Блок-схема функции ISR(INT0_vect)”

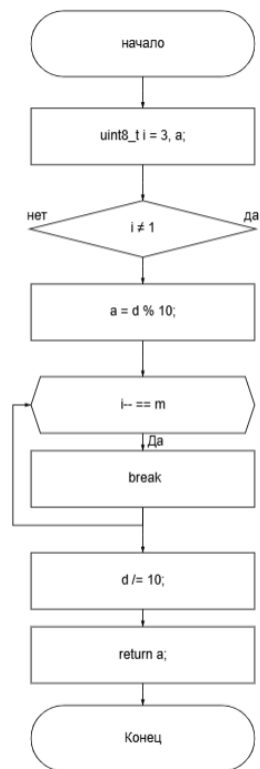


Рисунок 3 – “Блок-схема функции digit(uint16_t d, uint8_t m)”

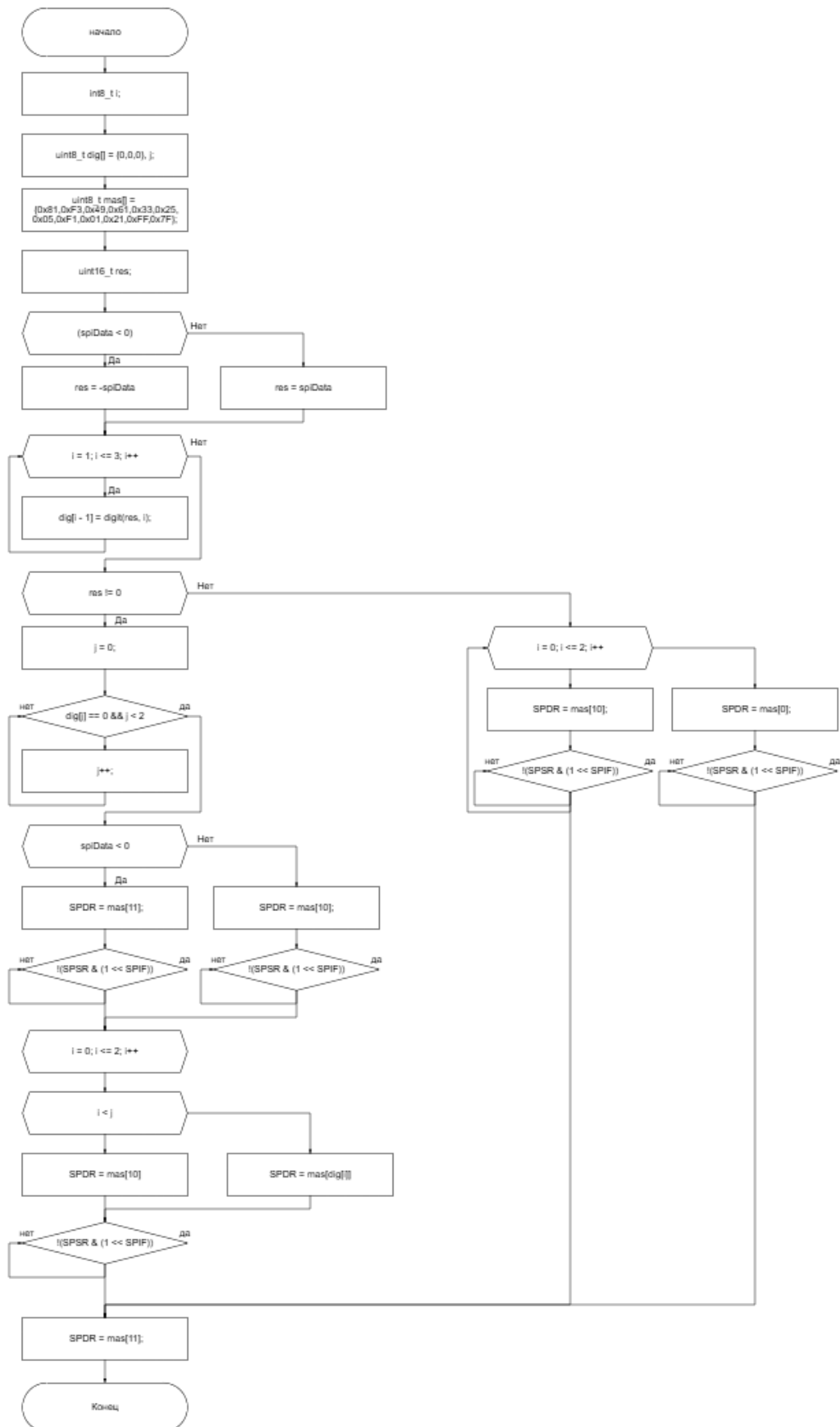


Рисунок 4 – “Блок-схема функции showMe(int16_t spiData)”



Рисунок 5 – “Блок-схема функции main(void)”

Выводы

В ходе выполнения данной лабораторной работы произошло ознакомление с протоколом SPI. Была реализована программа, выводящая на семисегментный модуль счетчик чисел от -100 до 100 (в нейтральном положении энкодера выводится 0, при повороте в одну сторону выводятся значения от 1 до 100, при повороте же в обратную сторону выводятся значения от -1 до -100).